

Parallax — Build Spec

Sep 25, 2026 · @Teclusion

Overview

Parallax is a credit card decisioning platform whose headline feature replays every past decision under a proposed rule change before it ships. It takes an application, pulls a credit report over SOAP, screens for fraud, runs policy rules and a scorecard, and returns approve, refer or decline with a limit and reason codes. Every decision lands in a tamper-evident ledger with its full inputs, so any decision can be reproduced exactly and replayed under new rules.

Why the name: parallax is seeing the same object from two positions. Parallax does that for credit decisions: every application viewed under the live rules and a candidate version, side by side, before anything ships.

Target role: Synchrony API Engineer 1 – Credit (L08), Hyderabad. The Credit team decisions accounts from acquisition through collections and manages credit and fraud losses. Parallax is built around that job.

JD requirement	Where Parallax covers it
1+ year applied Java	Java 21 across all modules
Spring / Spring Boot	Every service; Spring Security, Spring WS, Spring AI
SOAP, REST, JSON, XML web services	REST APIs; SOAP credit bureau with WSDL and XML mapping
Unit and integration tests	JUnit 5, Mockito, jqwik, Testcontainers, WireMock, golden-set replay
Continuous integration	GitHub Actions and Bitbucket Pipelines
Jira, Bitbucket, git	Jira board, issue keys in commits, Bitbucket mirror
Improve system performance	Strategy Lab replay of 100k decisions, measured and documented

JD requirement	Where Parallax covers it
Front-end UI	React app in Phase 3
Basic understanding of AI	Underwriter agent with tool calling, constrained to engine output
AI agents, tools, skills, instruction markdown	Spring AI agent, MCP server, AGENTS.md, Copilot instructions, skills folder
Credit and fraud losses	Fraud stage, exposure-weighted loss estimates, PSI drift monitoring

Architecture

The core idea is a pure decision engine packaged as a plain Java library, used both by a live service and by the replay tooling. Everything with I/O lives around it, never inside it.

```
flowchart LR
    C[Client / UI] -->|REST + Idempotency-Key| A[application-service]
    A -->|SOAP| B[bureau-mock]
    A -->|REST| D[decision-service]
    D --- E["(parallax-engine\npure library)"]
    A --- E
    A --> P["(Postgres:\napplications, ledger,\nrule versions)"]
    G["assistant-service\nSpring AI agent + MCP"] -->|read-only REST| A
    X["account-service\nPhase 3"] -->|REST| A
```

Modules

Module	Type	Responsibility
parallax-engine	Plain Java library, no Spring, no I/O	Fraud rules, policy rules, scorecard, limit, reason codes, rule-config validation
application-service	Spring Boot	Intake, idempotency, orchestration, bureau reuse window, ledger, REFER queue, Strategy Lab, PSI job
decision-service	Spring Boot, thin	Wraps the engine for live decisions over REST

Module	Type	Responsibility
bureau-mock	Spring Web Services	SOAP credit bureau with WSDL; simulates an external vendor
assistant-service	Spring Boot + Spring AI	Underwriter agent and MCP server; calls application-service with a read-only token
account-service	Spring Boot, Phase 3	Accounts, credit line increases, collections
web	React, Phase 3	Underwriter and strategist UI

Why the engine is a library: the Strategy Lab replays decisions next to the ledger data, in-process. The engine module has no Spring or database dependencies, so "pure and stateless" is enforced by the build, not claimed. Live decisions and replays run the same artifact; the reproduce endpoint catches any version skew.

Why the agent is its own service: its read-only access is enforced by the token it holds, not by its prompt. Even a manipulated model cannot call a write endpoint.

Deliberately left out: Kafka (synchronous orchestration is clearer and enough), Redis (the bureau reuse window is a business rule stored in Postgres), and an API gateway (little interview value versus the Strategy Lab).

Live decision flow

1. Client sends `POST /api/v1/applications` with an `Idempotency-Key` header.
2. Idempotency record inserted as `IN_PROGRESS` under a unique constraint on (client, key).
3. Application saved as `RECEIVED`.
4. Bureau: reuse a report pulled within the reuse window, otherwise a SOAP pull through a circuit breaker. The pull is stored and linked.
5. Derived features computed with I/O: velocity counts for SSN, email and phone over 24 hours.
6. The normalized `EngineInput` is sent to decision-service, which returns the decision and the rule version used.
7. One transaction: ledger record written, application set to `DECIDED`, idempotency record set to `COMPLETED` with the stored response.

8. Response returned. If a challenger version is in shadow mode, it is evaluated in-process and logged, never returned.

Tech stack

Area	Choice
Language	Java 21
Framework	Spring Boot 3, Spring Security, Spring Web Services, Spring AI
Data	PostgreSQL, Flyway, Spring Data JPA
Resilience	Resilience4j (circuit breaker, timeout, retry)
Testing	JUnit 5, Mockito, jqwik, Testcontainers, WireMock
Docs and ops	springdoc-openapi, Spring Actuator, Micrometer
Build and run	Maven multi-module, Docker Compose
CI	GitHub Actions, Bitbucket Pipelines
UI	React + TypeScript (Phase 3)

Core features

Every live decision passes through five stages: intake, bureau, fraud, policy and scorecard, then the ledger. Each stage below says what it does and the detail that makes it realistic.

1. Application intake

- `POST /api/v1/applications` with Bean Validation; field errors come back as a structured 400.
- **Idempotency:** unique constraint on (client_id, key), plus a hash of the request body.
 - Same key, same body, completed: return the stored response.
 - Same key, different body: 422.
 - Same key still `IN_PROGRESS` (a concurrent duplicate): 409.
 - Keys expire after 24 hours and are scoped per client.

2. Credit bureau (SOAP)

- bureau-mock exposes a WSDL; the adapter maps the XML report into an internal model.
- Returns open tradelines, 6-month inquiries, 24-month delinquencies, revolving utilization, file address, and SSN issuance year.
- Synthetic SSN ranges map to profiles (prime, near-prime, subprime, thin file) for demos.
- **Reuse window, not a cache:** a report pulled within N days (config, default 30) is reused. Every decision records which pull it used.
- **Pull type:** `SOFT` for prequalification, `HARD` for a real application.

3. Fraud and identity stage

Runs before policy, because identity verification (CIP/KYC) is a legal requirement for opening accounts. Any hit routes to REFER with a fraud reason code.

Check	Rule	Data source
Address mismatch	Application address differs from the bureau file address	Bureau report
SSN issuance	SSN issuance year is before the birth year	Bureau report (issuance year field)
Deceased SSN	SSN appears on a mock death master list	Seeded table
Velocity	Same SSN, email or phone on 3+ applications in 24 hours	Indexed query, computed before the engine runs

Fraud reasons never appear in applicant-facing text, as at real lenders.

4. Decision engine

Policy checks (knockouts):

- **Legal capacity:** applicant is 18+. Applicants under 21 need independent income or a co-signer (CARD Act).
- **Age is not a scoring factor.** Reg B permits age in an empirically derived system if older applicants are not scored lower; excluding it is a conservative design choice.

- **Bureau consent** is present.
- **Ability to pay (Reg Z 1026.51)**: monthly income minus obligations must cover the estimated minimum payment on the proposed limit, assuming full use. The threshold is a policy parameter, not a regulatory number.

Scorecard: each attribute falls into a band worth points; points sum to a score on a 300–850 scale. Attributes include utilization, inquiries, delinquencies, tradeline count, file age and income band.

Decision and limit: score cutoffs give APPROVED, REFER or DECLINED. The limit is the lower of the score-band limit and the ability-to-pay maximum.

Reason codes: up to 4. A failed policy rule comes first, then scorecard attributes ranked by points lost versus the maximum, each mapped to a standard adverse-action code.

Rule versions: rules are versioned config, loaded as data.

- A new version is validated on load: bands must not overlap or leave gaps, and cutoffs must be ordered.
- A version becomes immutable once any decision has used it.

5. Adverse action notices

Generated from a deterministic template filled with the reason codes, never by an LLM. Any AI wording is optional supplementary text for internal users.

6. Decision ledger

- One append-only record per decision with the **full normalized engine input**: bureau attributes, income, debt, housing cost, age at decision date, velocity counts.
- Identity fields (name, SSN, full DOB, contact details) are masked or tokenized; decision features are not.
- Also stores the rule version, scorecard version, bureau pull ID, output, reason codes and timestamp.
- **Enforced append-only**: the application's DB role has no UPDATE or DELETE grant on the ledger table.
- **Hash chain**: each record stores the hash of the previous one. Inserts are serialized with a Postgres advisory lock so concurrent decisions cannot fork the chain.
- **Reproduce**: `GET /api/v1/decisions/{id}/reproduce` re-runs the stored input under the recorded version and asserts an identical result.

7. Application states and failure paths

State	Meaning	Next
RECEIVED	Validated and saved	BUREAU_PULLED or BUREAU_UNAVAILABLE
BUREAU_PULLED	Report attached	DECIDED or ENGINE_FAILED
BUREAU_UNAVAILABLE	Circuit open or timeout; outcome REFER with an outage reason	Auto re-decision when the circuit closes
ENGINE_FAILED	Decision call failed	Retried with backoff, then manual queue
DECIDED	Ledger record and status written in one transaction	REVIEWED if the outcome was REFER
REVIEWED	Underwriter override recorded	Terminal

The audit write and the `DECIDED` status commit together. A decision that cannot be audited never exists.

8. REFER review queue

- Underwriters approve or decline a REFER with a note and an override reason code.
- Each override is its own ledger record, linked to the original decision.
- Override rate by score band is reported, so a strategist can spot a miscalibrated band.

9. Resilience

- Resilience4j circuit breaker and timeout around the SOAP call.
- Demo: stop the bureau container, submit applications, watch them go to REFER, restart it, watch the re-decision job clear them.

10. Security

Role	Can do
CLIENT	Submit applications, read its own decisions

Role	Can do
UNDERWRITER	Work the REFER queue, read decisions (identity masked)
STRATEGIST	Create rule versions, run replays, propose promotions
APPROVER	Approve or reject promotions
AUDITOR	Read the ledger with no access to raw PII
ASSISTANT	Read-only service account for the agent

- SSNs encrypted at rest with AES-GCM through a JPA attribute converter; the key comes from the environment.
- Log masking for SSN, DOB, email and phone via a Logback converter, with a test that fails if a raw SSN reaches the logs.

Strategy Lab (the headline feature)

A strategist proposes a rule version, replays 100k historical decisions under it, reads an honest impact report, and promotes it through a second approver. This is champion/challenger testing, the core workflow of a credit strategy team.

Workflow

1. **Draft:** a strategist creates a candidate version, for example raising the approve cutoff from 620 to 640. It must pass config validation.
2. **Replay:** an async job re-runs every ledger record in a date range through the candidate. `POST /api/v1/lab/replays` returns a job ID.
3. **Report:** the diff between the live version's recorded outcomes and the candidate's outcomes.
4. **Shadow (optional):** the candidate runs silently beside the live version on new applications and logs disagreements.
5. **Propose and approve:** maker-checker. The proposer cannot approve their own change.
6. **Promote or roll back:** promotion makes the candidate live; rollback restores the previous version in one action. Both are audited.

The diff report

- Approval rate before and after, and the count of flips in each direction (approve→decline, decline→approve, and to or from REFER).
- Drill-down into flipped cases with old and new reason codes side by side.
- **Segment breakdown** by score band, income band and product. Strategists never judge only the headline number.
- **Limit changes** even where the outcome did not flip.
- **Override signal**: REFER override rates by band, from the review queue.

Expected loss, done properly

Loss is exposure-weighted, not a default rate:

$$\mathrm{EL} = \mathrm{PD} \times \mathrm{EAD} \times \mathrm{LGD}, \quad \mathrm{EAD} \approx \mathrm{Limit} \times \mathrm{CCF}$$

- PD comes from observed synthetic outcomes where available, otherwise from the score band's default rate.
- CCF (drawdown factor) and LGD are config parameters, stated as assumptions in the report.
- A rule that only changes limits still moves expected loss, and the report shows it.

Reject inference (the honest caveat)

Outcomes exist only for applicants who were approved. Tightening a cutoff can be measured on observed data; loosening one approves people whose outcomes were never observed.

- Newly approved cases are labelled **outcome unknown** and excluded from observed-loss totals.
- An optional **simulated** view uses the generator's counterfactual outcomes for declines. It is clearly labelled as a simulation.

Performance

- The engine is CPU-bound and pure, so evaluation runs on a bounded parallel executor sized to the CPU count.
- The real cost is loading data: stream ledger rows with a JDBC fetch size and keyset pagination rather than loading entity graphs.

- Target to measure and publish in the README: 100k replays, end to end, with the time split between load and evaluate.

Drift monitoring (PSI)

A scheduled job compares the current score distribution against the development baseline using the Population Stability Index. The usual reading is below 0.1 stable, 0.1–0.25 watch, above 0.25 investigate. Alerts surface in the lab and through Actuator metrics.

Synthetic data generator

- Produces around 100k historical applicants with bureau attributes, income and 12-month outcomes.
- Default probability is driven by utilization, delinquencies, inquiries and debt burden through a logistic function, so outcomes correlate sensibly with score.
- Generates counterfactual outcomes for declined applicants, used only by the simulated view.
- Seeded for reproducibility, so reports and CI golden sets are stable.

AI: agent, tools and instruction files

The agent reads and explains; it never makes or changes a decision. That constraint is enforced by its credentials, its tool list and its tests, not only its prompt.

Underwriter and strategist agent

Built with Spring AI tool calling in assistant-service. Example questions:

- "Why was APP-1042 declined?"
- "Why did approvals drop under v1.4?"
- "Which score band has the highest REFER override rate?"

Tool	Returns	Guard
<code>getDecision(id)</code>	Outcome, score, limit, version, identity masked	Read-only
<code>getReasonCodes(id)</code>	Codes with standard descriptions	Read-only

Tool	Returns	Guard
<code>runReplay(candidateVersion)</code>	Job ID, or cached results if already replayed	Existing versions only; async; rate-limited
<code>getReplayReport(jobId)</code>	Diff report summary and segments	Read-only
<code>compareVersions(a, b)</code>	Rule-by-rule config diff	Read-only

Guardrails

- **Credentials:** the service account has the ASSISTANT role, which has no write permissions anywhere.
- **Grounding:** every number in an answer must come from a tool result; the system prompt says so and the evals check it.
- **Prompt injection:** applicant-supplied text (names, addresses) flows back through tool results. Tool output is framed as data, and a test submits an application whose address contains instructions and asserts the agent ignores them.
- **PII:** tools return masked identity fields, so raw SSNs and DOBs never reach the model.
- **Not for applicants:** adverse action notices stay on the deterministic template.

Evals

A set of 15 questions with known answers from seeded data, run in CI against the agent. Each eval checks the answer's facts and the tools called. Results are tracked over time in the repo.

MCP server (stretch)

The same tools exposed over the Model Context Protocol with Spring AI's MCP server support, under the same read-only role. Claude or Copilot can then query Parallax directly from an editor.

Instruction files and skills

File	Contents
<code>AGENTS.md</code> / <code>CLAUDE.md</code>	Module map, conventions, invariants: never log PII, the engine stays pure, ledger is append-only, every rule change needs tests
<code>.github/copilot-instructions.md</code>	The same conventions in Copilot's format
<code>skills/add-scorecard-attribute/SKILL.md</code>	Steps: config, validation, engine, reason-code mapping, property tests, golden-set update
<code>skills/add-policy-rule/SKILL.md</code>	Steps for a new knockout rule with tests and a reason code
<code>skills/run-strategy-replay/SKILL.md</code>	How to draft a version, replay it and read the report
<code>docs/ai-workflow.md</code>	How the project was built with Copilot or Claude Code, including at least one thing the AI got wrong and how it was caught

Keep these short and current. An instruction file that no longer matches the code is worse than none.

Data model

Nine tables in application-service's database carry the design. The ledger schema is the one that must be right in Phase 1, because replay depends on it.

Table	Key columns	Invariants
<code>application</code>	<code>id</code> , <code>client_id</code> , <code>status</code> , <code>ssn_encrypted</code> , <code>ssn_token</code> , <code>name_masked</code> , <code>email_hash</code> , <code>phone_hash</code> , <code>created_at</code>	SSN encrypted at rest; status changes only along allowed transitions
<code>idempotency_key</code>	<code>client_id</code> , <code>key</code> , <code>request_hash</code> , <code>state</code> , <code>response_body</code> , <code>expires_at</code>	Unique (<code>client_id</code> , <code>key</code>)

Table	Key columns	Invariants
<code>bureau_pull</code>	<code>id</code> , <code>ssn_token</code> , <code>pull_type</code> , <code>pulled_at</code> , <code>attributes (jsonb)</code> , <code>raw_xml_ref</code>	Reuse only within the configured window and matching pull type
<code>rule_version</code>	<code>version</code> , <code>config (jsonb)</code> , <code>status</code> , <code>created_by</code> , <code>approved_by</code> , <code>first_used_at</code>	Validated on insert; immutable once <code>first_used_at</code> is set
<code>decision_ledger</code>	<code>id</code> , <code>application_id</code> , <code>engine_input (jsonb)</code> , <code>rule_version</code> , <code>scorecard_version</code> , <code>bureau_pull_id</code> , <code>outcome</code> , <code>score</code> , <code>limit</code> , <code>reason_codes</code> , <code>prev_hash</code> , <code>hash</code> , <code>created_at</code>	No UPDATE or DELETE grant; hash-chained; inserts serialized
<code>review</code>	<code>id</code> , <code>ledger_id</code> , <code>underwriter</code> , <code>outcome</code> , <code>override_code</code> , <code>note</code>	Creates a linked ledger record
<code>replay_job</code>	<code>id</code> , <code>candidate_version</code> , <code>date_from</code> , <code>date_to</code> , <code>status</code> , <code>report (jsonb)</code> , <code>timings</code>	One running job per candidate
<code>shadow_result</code>	<code>ledger_id</code> , <code>candidate_version</code> , <code>outcome</code> , <code>score</code> , <code>limit</code> , <code>agrees</code>	Never returned to clients
<code>loan_outcome</code>	<code>application_id</code> , <code>defaulted</code> , <code>months_observed</code> , <code>exposure</code> , <code>simulated</code>	Simulated rows only feed the labelled simulation view

Indexes worth calling out: `(ssn_token, created_at)`, `(email_hash, created_at)` and `(phone_hash, created_at)` for velocity; `(created_at, id)` on the ledger for keyset-paginated replay.

engine_input holds the normalized feature vector exactly as the engine saw it: bureau attributes, income, debt, housing cost, age at decision date, velocity counts and pull type. Identity fields are never in it.

Engineering practices

Tests are weighted toward the engine, where correctness matters most, and CI runs on both GitHub and Bitbucket.

Testing

Layer	Tool	What it proves
Engine unit tests	JUnit 5	Each rule, band and reason-code ranking
Engine property tests	jqwik	Lowering utilization never lowers the score; the limit never exceeds the ability-to-pay maximum; at most 4 reason codes; same input, same output
Rule config validation	JUnit 5	Overlapping bands, gaps and misordered cutoffs are rejected
Service unit tests	Mockito	Orchestration, idempotency branches, state transitions
Integration	Testcontainers (Postgres)	Transactional audit write, ledger grants, hash chain under concurrent inserts, concurrent idempotency (409)
Bureau	WireMock + one Spring WS contract test	Adapter mapping, timeouts, circuit breaker; the real WSDL is exercised
Golden set	CI job	500 seeded decisions reproduce identically
Log safety	JUnit 5	A raw SSN never reaches log output
Agent evals	CI job	15 questions answered correctly with the expected tools

CI

- **GitHub Actions** (source of truth): build, unit tests, integration tests, golden set, coverage report on every PR.
- **Bitbucket**: push mirror plus `bitbucket-pipelines.yml` running build and tests.
- Agent evals run nightly or on changes to assistant-service, since they call an LLM.

Jira and git

- Free Jira board with epics per phase and user stories per feature.
- Branches like `feature/PX-42-velocity-check` ; commit messages start with the issue key, which links Jira, Bitbucket and git.
- Small PRs, each with tests.

Repository layout

```

parallax/
  AGENTS.md
  CLAUDE.md
  .github/copilot-instructions.md
  .github/workflows/ci.yml
  bitbucket-pipelines.yml
  docker-compose.yml
  parallax-engine/           pure Java library
  application-service/
  decision-service/
  bureau-mock/
  assistant-service/
  account-service/           Phase 3
  data-generator/
  web/                       Phase 3
  skills/
  docs/ai-workflow.md
  docs/adr/                  architecture decision records

```

`docker compose up` starts Postgres and all services with seeded data. The README leads with a 60-second demo script.

Phase plan

Phase 1 fixes everything that is expensive to change later; each phase after it is shippable on its own. Estimates assume part-time work of roughly 15 hours a week.

Phase	Scope	Estimate	Done when
1. Foundation	Engine library, application-service, decision-service, SOAP bureau, fraud stage (4 checks), full-input ledger with enforced append-only, transactional audit, failure states, concurrency-safe idempotency, two-role auth, SSN encryption, log masking, reproduce endpoint, Flyway, OpenAPI, Actuator, tests, Docker Compose, GitHub CI, AGENTS.md	3–4 weeks	A demo applicant goes from submit to reproducible decision; golden set passes in CI
2. Core	Data generator, Strategy Lab (replay, exposure-weighted loss, reject-inference labels, segments), hash chain, full roles, REFER queue, resilience and re-decision, agent with evals, Bitbucket mirror, skills folder	3 weeks	100k replay timed and published; agent passes evals
2. Stretch	Shadow mode, maker-checker promotion and rollback, PSI job, MCP server	1–2 weeks	Each item demoable in isolation
3. Surface	React UI, account-service (accounts, credit line increase, collections)	2–3 weeks	UI covers submit, decision detail, REFER queue, Strategy Lab

If time runs short, cut from the bottom: Phase 3 first, then Phase 2 stretch. Never cut the ledger schema, the reproduce endpoint or the reject-inference labelling; they are the design's proof.

Interview talking points

Each design decision has a one-line reason you should be able to say without notes.

Likely question	Your answer
Why is the engine a separate library?	Replay needs the exact same logic in-process; no Spring or I/O means purity is enforced by the build.
How do you reproduce a decision from six months ago?	The ledger stores the full input and the rule version; the reproduce endpoint re-runs it, and CI checks 500 of them.
What if the audit write fails?	It is in the same transaction as the status change, so an unaudited decision cannot exist.
What stops a double submit?	A unique constraint on the idempotency key; a concurrent duplicate gets 409, a mismatched body gets 422.
What happens when the bureau is down?	The circuit opens, applications go to REFER with an outage reason, and a job re-decides them when it closes.
Why not just cache bureau reports?	A pull is a point-in-time snapshot and hard pulls affect the applicant, so reuse is a configurable business rule.
How do you know the replay loss numbers are honest?	Loss is $PD \times EAD \times LGD$, and newly approved applicants are marked outcome unknown because of reject inference.
Why is age not in the scorecard?	Reg B allows it under strict conditions; leaving it out is a conservative choice. Legal capacity and the under-21 rule are still checked.
Why is the replay fast?	Evaluation is pure CPU work on a bounded executor; the real work was streaming rows efficiently from Postgres.
How is the AI agent kept safe?	Read-only credentials, tool outputs only, masked PII, an injection test, and evals in CI.

Likely question	Your answer
How did you use AI tools to build this?	Point to AGENTS.md, the skills folder and docs/ai-workflow.md , including the mistake you caught.
How would a rule change reach production?	Draft, validate, replay, optional shadow, maker-checker approval, promote, with one-click rollback.